The Java Algorithmic Compendium: Mastering Comparators, PriorityQueues, Functional Idioms, and Modern Collections APIs

Abstract

A definitive, mathematically rigorous, and code-first reference manual for software engineers and competitive programmers preparing for top-tier technical interviews. This treatise systematically demystifies modern Java functional syntax, lambdas, and built-in collection primitives that frequently cause confusion in algorithmic problem solving. We deconstruct: (1) The anatomical mechanics of the java.util.Comparator functional interface, the three-way comparison contract, and the fatal integer-overflow pitfall of naive subtraction; (2) The four evolutionary stages of Java comparison syntax, culminating in modern fluent factory combinators like Comparator.comparingInt(); (3) Comprehensive PriorityQueue (Binary Heap) mastery, encompassing min-heaps, max-heaps, multi-tier tie-breaking comparators, and the bounded Top-K pattern; (4) Primitive array vs. Object array sorting pitfalls and the generic type-erasure boundary; (5) The essential modern Collections APIs (computeIfAbsent, merge, ArrayDeque, binary search insertion-index mechanics); and (6) Battle-tested reference implementations for canonical algorithmic archetypes (Merge K Sorted Lists, Top K Frequent Words, Interval Scheduling, and Skyline Sweepline).

Index Terms

Java, Comparator, PriorityQueue, Min-Heap, Max-Heap, Functional Programming, Lambdas, Method References, Algorithmic Engineering, LeetCode

1 Introduction: The Mental Model of Java Comparisons

In technical interviews and competitive programming, candidates frequently stumble over the exact syntax for custom sorting and priority queues. Consider the idiomatic snippet:

```
PriorityQueue<int[]> minHeap = new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));
```

To an engineer unfamiliar with Java 8+ functional interfaces, this line resembles magical boilerplate. However, it is rooted in a precise, logical mathematical model.

The goal of this compendium is to permanently eliminate hesitation with Java built-in methods, replacing rote memorization with first-principles understanding.

2 The Core Comparison Contract: How Java Compares Objects

Whenever Java sorts an array, rebalances a TreeSet, or sifts an element up a PriorityQueue, it relies on a single question: "Does element A belong before element B, after B, or are they equivalent?"

2.1 The Three-Way Comparison Contract

A comparator is formalized as a function $f : (T, T) \to \mathbb{Z}$ adhering to:

$$\text{compare}(a, b) = \begin{cases} \text{negative integer } (< 0), & \text{if } a < b \quad (a \text{ belongs before } b \text{ in ascending order}) \\ 0, & \text{if } a == b \quad (\text{equivalent ordering}) \\ \text{positive integer } (> 0), & \text{if } a > b \quad (a \text{ belongs after } b \text{ in ascending order}) \end{cases} \tag{1}$$

The Golden Memory Anchor:

"If the result is negative, keep the order (a, b). If the result is positive, swap the order to (b, a)."

2.2 The Catastrophic Integer Subtraction Trap

Junior candidates frequently write:

```
// DANGEROUS ANTI-PATTERN: DO NOT USE IN PRODUCTION OR INTERVIEWS
(a, b) -> a[1] - b[1]
```

Why this causes silent failures: If $a[1] = -2{,}000{,}000{,}000$ and $b[1] = 2{,}000{,}000{,}000$, mathematical subtraction yields:

$$-2{,}000{,}000{,}000 - 2{,}000{,}000{,}000 = -4{,}000{,}000{,}000 \tag{2}$$

In standard 32-bit signed two's-complement arithmetic, this underflows into positive +294,967,296. The comparator returns a positive integer, completely reversing the sort order and corrupting your heap!

The Iron Rule: Always use Integer.compare(a, b) or Comparator.comparingInt(). They perform sign checks internally without arithmetic overflow:

```
// SAFE AND ROBUST
(a, b) -> Integer.compare(a[1], b[1])
```

3 The 4 Evolutions of Java Comparator Syntax

To master Comparator.comparingInt(a -> a[1]), observe how Java evolved across four syntax generations for sorting 2D interval arrays by start time a[0]:

3.1 Evolution 1: The Anonymous Inner Class (Java 1.2 to 7)

```
PriorityQueue<int[]> pq = new PriorityQueue<>(new Comparator<int[]>() {
    @Override
    public int compare(int[] a, int[] b) {
        return Integer.compare(a[0], b[0]);
    }
});
```

Verdict: Verbose, noisy boilerplate, but illustrates the underlying interface contract.

3.2 Evolution 2: The Explicit Two-Argument Lambda (Java 8+)

Because Comparator<T> is a Functional Interface with exactly one abstract method (compare), the compiler infers the interface type:

```
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> Integer.compare(a[0], b[0]));
```

Verdict: Clean, direct, and universally accepted in competitive programming.

3.3 Evolution 3: The Functional Key Extractor — Comparator.comparingInt

Instead of writing the comparison logic yourself, modern Java provides high-order combinator functions. You simply tell Java: "Here is a function that extracts an integer key from my object. Compare elements based on that key."

```
PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[0]));
```

Anatomy Breakdown:

1) a -> a[0]: A lambda function (implements ToIntFunction<int[]>) that accepts array a and extracts the primitive int key at index 0.
2) Comparator.comparingInt(...): A static factory method that accepts this key extractor and produces a fully optimized, zero-boxing Comparator<int[]>.

3.4 Evolution 4: Method References (Cleanest for Objects)

When sorting custom objects with getters, the lambda can be replaced with a method reference:

```
class Interval { int start; int end; public int getStart() { return start; } }

// Modern Declarative Syntax:
PriorityQueue<Interval> pq = new PriorityQueue<>(Comparator.comparingInt(Interval::getStart));
```

4 PriorityQueue Mastery: Min-Heaps, Max-Heaps, and Tie-Breakers

A Java PriorityQueue is a min-heap by default. The root of the heap (pq.peek()) is the smallest element according to the specified comparator.

4.1 Min-Heap vs. Max-Heap Cheat-Sheet

The following four archetypes cover 100% of heap declarations needed in algorithmic interviews:

1) Integer Min-Heap (Natural Ascending Order):

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
```

2) Integer Max-Heap (Descending Order):

```
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
// Or via explicit lambda:
PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> Integer.compare(b, a));
```

3) 2D Array Min-Heap (Ordered by index 1 ascending):

```
PriorityQueue<int[]> minHeap = new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));
// Or via explicit lambda:
PriorityQueue<int[]> minHeap = new PriorityQueue<>((a, b) -> Integer.compare(a[1], b[1]));
```

4) 2D Array Max-Heap (Ordered by index 1 descending):

```
PriorityQueue<int[]> maxHeap = new PriorityQueue<>((a, b) -> Integer.compare(b[1], a[1]));
// Or via modern combinator:
PriorityQueue<int[]> maxHeap = new PriorityQueue<>(
    Comparator.comparingInt((int[] a) -> a[1]).reversed()
);
```

4.2 Multi-Criteria Sorting (Tie-Breakers)

In many classic problems (e.g., LeetCode 692: Top K Frequent Words), you must sort by multiple criteria:

> "Sort words by frequency in descending order; if frequencies are identical, sort alphabetically in ascending order."

4.2.1 Approach A: The Idiomatic Lambda (Fastest in Interviews)

```
PriorityQueue<Map.Entry<String, Integer>> pq = new PriorityQueue<>((a, b) -> {
    if (!a.getValue().equals(b.getValue())) {
        return Integer.compare(b.getValue(), a.getValue()); // Descending Frequency (Max-Heap)
    }
    return a.getKey().compareTo(b.getKey());              // Ascending Alphabetical (Min-Heap)
});
```

4.2.2 Approach B: The Modern Fluent Combinator (thenComparing)

```
Comparator<Map.Entry<String, Integer>> comp = Comparator
    .<Map.Entry<String, Integer>>comparingInt(Map.Entry::getValue).reversed()
    .thenComparing(Map.Entry::getKey);

PriorityQueue<Map.Entry<String, Integer>> pq = new PriorityQueue<>(comp);
```

Rule of Thumb: In high-pressure coding interviews, Approach A (the explicit lambda) is easier to reason through without fighting generic type inference.

4.3 The Bounded Top-K Optimization Pattern

To find the K largest elements across a stream of N items:

- Mistake: Store all N items in a Max-Heap $\implies$ $O(N \log N)$ time, $O(N)$ space.
- Optimal Pattern: Maintain a Min-Heap of size K $\implies$ $O(N \log K)$ time, $O(K)$ space!

```
// Bounded Min-Heap of size K for K Largest Elements
PriorityQueue<Integer> minHeap = new PriorityQueue<>(k);
for (int num : nums) {
    minHeap.offer(num);
    if (minHeap.size() > k) {
        minHeap.poll(); // Evicts the smallest; top K largest remain in the heap!
    }
}
```

5 Array Sorting Pitfalls: Why int[] Fails with Comparators

A frequent source of compilation errors is attempting to pass a custom comparator to a 1D primitive array:

```
int[] arr = {5, 2, 8, 1};
Arrays.sort(arr, (a, b) -> b - a); // COMPILATION ERROR!
```

5.1 Why Does This Happen?

Java generics do not support primitive types (int, double, char). The method signature is:

public static <T> void sort(T[] a, Comparator<? super T> c)

Because int is a primitive, int[] is treated as a single object, not an array of objects T[].

5.2 The Three Correct Workarounds

1) Box to Integer[]:

```
Integer[] boxed = Arrays.stream(arr).boxed().toArray(Integer[]::new);
Arrays.sort(boxed, Collections.reverseOrder());
```

2) Sort Ascending then Invert in-place (Fastest, zero allocation):

```
Arrays.sort(arr);
for (int i = 0, j = arr.length - 1; i < j; i++, j--) {
    int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
}
```

3) 2D Array (Works naturally because int[][] is an array of int[] objects):

```
int[][] intervals = {{1, 3}, {2, 6}, {8, 10}};
Arrays.sort(intervals, Comparator.comparingInt(a -> a[0])); // COMPILES PERFECTLY!
```

6 The Essential Modern Java Collections API Cheat-Sheet

Top-percentile engineers avoid messy if (map.containsKey()) chains by leveraging modern collection utilities introduced in Java 8 and 11.

TABLE 1
High-Frequency Built-In Methods for Algorithmic Interviews

Category	Method Signature	Algorithmic Behavior & Clean-Code Purpose
Map Frequency	map.getOrDefault(key, 0) + 1	Safe retrieval with fallback default; eliminates null checks.
Map Frequency	map.merge(key, 1, Integer::sum)	Atomically inserts 1 if absent, or adds 1 to existing value.
Map Grouping	map.computeIfAbsent(key, k -> new ArrayList<>())	Retrieves existing list or initializes new list and stores it in map.
Queue / Stack	Deque<Integer> stack = new ArrayDeque<>()	Modern replacement for Stack; use push(), pop(), peek().
Binary Search	int idx = Arrays.binarySearch(arr, target)	Returns index if found; returns -(insertion_point + 1) if missing.
PriorityQueue	pq.offer(x), pq.poll(), pq.peek()	Preferred over add()/remove() because they return special values on error.
Char Checks	Character.isLetterOrDigit(c)	Eliminates ASCII range checks in palindrome/string parsing.
String Concatenation	String.join(",", list)	Assembles collections into delimited strings without trailing commas.

6.1 Deep Dive: The Binary Search Insertion-Point Formula

When Arrays.binarySearch(arr, key) returns a negative value V:

$$\text{insertion_index} = -(V + 1) \tag{3}$$

Example: Array = [10, 20, 30]. Search for 25.

- 25 should belong at index 2 (between 20 and 30).
- binarySearch returns -3.
- insertion_index $= -(-3 + 1) = 2$.

7 The Fatal TreeSet & TreeMap Equality Trap

Many candidates confuse PriorityQueue with TreeSet. Both accept custom comparators, but they behave completely differently when two elements are deemed equivalent:

- In PriorityQueue: If compare$(a, b) == 0$, both elements remain in the heap. Duplicates are fully preserved.
- CRITICAL TRAP in TreeSet & TreeMap: A TreeSet does NOT use equals() to check for duplicates! It relies solely on your Comparator. If your comparator returns 0, TreeSet silently discards the incoming element as a duplicate!

7.1 The Bug and the Production Defense

```
// DANGEROUS BUG: Any two intervals with the same start time will be DELETED!
TreeSet<int[]> set = new TreeSet<>((a, b) -> Integer.compare(a[0], b[0]));
set.add(new int[]{1, 5});
set.add(new int[]{1, 9}); // SILENTLY DROPPED! set.size() is still 1!

// BULLETPROOF DEFENSE: Always add a secondary unique tie-breaker (e.g., end time or ID):
TreeSet<int[]> safeSet = new TreeSet<>((a, b) -> {
   if (a[0] != b[0]) return Integer.compare(a[0], b[0]);
   return Integer.compare(a[1], b[1]); // Tie-breaker preserves distinct intervals!
});
```

7.2 TreeSet Navigation Superpowers

- set.ceiling(x): Smallest element $\geq x$ (Null if none).
- set.floor(x): Largest element $\leq x$ (Null if none).
- set.higher(x): Strictly greater $> x$.
- set.lower(x): Strictly smaller $< x$.

8 Rapid Type Conversions, Stream Idioms, and Bitwise Superpowers

In timed interviews, spending 5 minutes writing loops to convert data types is an immediate red flag. Master these essential one-liners:

8.1 1. Converting Between Arrays and Lists

```
// 1. List<Integer> -> int[]
int[] arr = list.stream().mapToInt(i -> i).toArray();

// 2. int[] -> List<Integer>
List<Integer> list = Arrays.stream(arr).boxed().collect(Collectors.toList());

// 3. String[] -> List<String>
List<String> strList = Arrays.asList(strArr);

// 4. char[] -> String (NEVER call charArr.toString(), which prints memory hash "[C@...")
String s = new String(charArr);

// 5. String -> char[]
char[] chars = s.toCharArray();
```

8.2 2. Matrix & 2D Array Fast Initialization

Never write nested loops to initialize a DP memoization table:

```
int[][] memo = new int[m][n];
for (int[] row : memo) Arrays.fill(row, -1); // Clean, idiomatic, and fast
```

8.3 3. Circular Array Indexing: Negative Modulo Defense

In circular arrays or string rotations, $(i - k) \pmod{n}$ in Java produces negative results if $i < k$!

```
// BAD: (-1 % 5) produces -1 (ArrayIndexOutOfBoundsException!)
// GOOD: Math.floorMod handles negative numbers correctly in cyclic buffers:
int safeIndex = Math.floorMod(i - k, n); // Math.floorMod(-1, 5) produces 4!
```

8.4 4. Bitwise Built-in Methods

- Integer.bitCount(n): Returns the number of 1-bits (Hamming weight) in $O(1)$ CPU cycles.
- Integer.highestOneBit(n): Returns the largest power of 2 less than or equal to n.
- Integer.toBinaryString(n): Returns binary string representation for rapid debugging.

8.5 5. StringBuilder Backtracking Protocol

When exploring paths in tree/graph backtracking:

```
StringBuilder sb = new StringBuilder();
sb.append(ch);              // 1. Make choice
backtrack(..., sb);         // 2. Recurse
sb.deleteCharAt(sb.length() - 1); // 3. Undo choice (O(1) pop)
```

9 Canon Algorithmic Blueprints: Production Reference Implementations

9.1 Blueprint 1: Merge K Sorted Lists (LeetCode 23)

```
public class ListNode { int val; ListNode next; }

public ListNode mergeKLists(ListNode[] lists) {
    if (lists == null || lists.length == 0) return null;

    // Min-Heap ordered by node value
    PriorityQueue<ListNode> pq = new PriorityQueue<>(Comparator.comparingInt(node -> node.val));

    // Seed the heap with head of each non-empty list
    for (ListNode head : lists) {
        if (head != null) pq.offer(head);
    }

    ListNode dummy = new ListNode();
    ListNode tail = dummy;

    while (!pq.isEmpty()) {
        ListNode minNode = pq.poll();
        tail.next = minNode;
        tail = tail.next;

        if (minNode.next != null) {
            pq.offer(minNode.next);
        }
    }
    return dummy.next;
}
```

9.2 Blueprint 2: Meeting Rooms II / Overlapping Intervals (LeetCode 253)

```
public int minMeetingRooms(int[][] intervals) {
    if (intervals == null || intervals.length == 0) return 0;

    // 1. Sort intervals by START time ascending
    Arrays.sort(intervals, Comparator.comparingInt(a -> a[0]));

    // 2. Min-Heap storing END times of currently occupied rooms
    PriorityQueue<Integer> occupiedRooms = new PriorityQueue<>();

    for (int[] meeting : intervals) {
        // If earliest meeting has already ended before this meeting starts, reuse room
        if (!occupiedRooms.isEmpty() && occupiedRooms.peek() <= meeting[0]) {
            occupiedRooms.poll();
        }
        // Allocate room with new meeting's end time
```

```
        occupiedRooms.offer(meeting[1]);
    }

    // Size of heap represents minimum rooms required
    return occupiedRooms.size();
}
```

9.3 Blueprint 3: K Closest Points to Origin (LeetCode 973)

```
public int[][] kClosest(int[][] points, int k) {
    // Max-Heap of size K: stores furthest point at the top so it can be evicted
    PriorityQueue<int[]> maxHeap = new PriorityQueue<>(
        (a, b) -> Integer.compare(b[0]*b[0] + b[1]*b[1], a[0]*a[0] + a[1]*a[1])
    );

    for (int[] p : points) {
        maxHeap.offer(p);
        if (maxHeap.size() > k) {
            maxHeap.poll(); // Evict furthest point
        }
    }

    int[][] result = new int[k][2];
    for (int i = 0; i < k; i++) {
        result[i] = maxHeap.poll();
    }
    return result;
}
```

10 The Catastrophic HashSet<int[]> Array Identity Trap

Perhaps the most insidious, silent bug in LeetCode graph traversals (e.g., tracking visited coordinates in a 2D grid) is storing primitive arrays in hash collections:

```
// CATASTROPHIC BUG: NEVER DO THIS!
Set<int[]> visited = new HashSet<>();
visited.add(new int[]{1, 2});

System.out.println(visited.contains(new int[]{1, 2})); // PRINTS FALSE!
```

10.1 Why Does This Return false?

In Java, arrays do not override equals() or hashCode(). They inherit the default implementation from java.lang.Object, which performs **identity (memory address) comparison**. Two distinct array instances in memory with identical values have completely different hash codes and are never equal according to equals().

10.2 The Three Production Solutions

1) 1D Flattened Index (Fastest, zero memory overhead): If the grid has dimensions $R \times C$:

   ```
   int key = row * C + col; // Guaranteed unique integer
   Set<Integer> visited = new HashSet<>();
   visited.add(key);
   ```

2) Modern Java Record (Cleanest, Java 14+): Java record components automatically generate deep, value-based equals() and hashCode() implementations:

   ```
   record Point(int r, int c) {}

   Set<Point> visited = new HashSet<>();
   visited.add(new Point(1, 2));
   visited.contains(new Point(1, 2)); // PRINTS TRUE!
   ```

3) Immutable List wrapper (Java 9+):

   ```
   Set<List<Integer>> visited = new HashSet<>();
   visited.add(List.of(1, 2));
   visited.contains(List.of(1, 2)); // PRINTS TRUE! (List implements value-based equality)
   ```

11 The Monotonic Queue Pattern (ArrayDeque)

To find sliding window maximums (LeetCode 239) in optimal $O(N)$ time, standard priority queues ($O(N \log K)$) are too slow. We maintain a Monotonically Decreasing Deque storing array indices:

```
public int[] maxSlidingWindow(int[] nums, int k) {
    if (nums == null || k <= 0) return new int[0];
    int n = nums.length;
    int[] result = new int[n - k + 1];
    int ri = 0;

    // Deque stores INDICES. Values at indices are strictly descending!
    Deque<Integer> deque = new ArrayDeque<>();

    for (int i = 0; i < n; i++) {
        // 1. Remove indices that have fallen outside the current window
        while (!deque.isEmpty() && deque.peekFirst() < i - k + 1) {
            deque.pollFirst();
        }

        // 2. Remove all elements smaller than current element (they can never be the max)
        while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) {
            deque.pollLast();
        }

        // 3. Add current element's index to back of deque
        deque.offerLast(i);

        // 4. Record current window maximum (located at the front)
        if (i >= k - 1) {
            result[ri++] = nums[deque.peekFirst()];
        }
    }
    return result;
}
```

12 The 10-Line Bulletproof Disjoint Set Union (DSU / Union-Find)

Union-Find is indispensable for graph cycle detection, connected components, and Kruskal's Minimum Spanning Tree. Memorize this clean, optimal template with **Path Compression** and **Union by Rank**:

```
public class DSU {
    private final int[] parent;
    private final int[] rank;

    public DSU(int n) {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    public int find(int i) {
        if (parent[i] == i) return i;
        return parent[i] = find(parent[i]); // Path compression: O(alpha(N)) amortized
    }

    public boolean union(int i, int j) {
        int rootI = find(i);
        int rootJ = find(j);
        if (rootI == rootJ) return false; // Cycle detected!

        // Union by rank
        if (rank[rootI] < rank[rootJ]) parent[rootI] = rootJ;
```

```
        else if (rank[rootI] > rank[rootJ]) parent[rootJ] = rootI;
        else {
            parent[rootJ] = rootI;
            rank[rootI]++;
        }
        return true;
    }
}
```

13 Radix Parsing, String Splitting, and Randomization Superpowers

- Binary & Radix Parsing:

```
int decimal = Integer.parseInt("1011", 2); // Converts binary String to 11
String binaryStr = Integer.toBinaryString(11); // Converts 11 to "1011"
String hexStr = Integer.toHexString(255);      // Converts 255 to "ff"
```

- Robust Whitespace String Splitting:

```
// Bad: str.split(" ") breaks on multiple spaces and leading whitespace!
// Good: Handles arbitrary consecutive whitespace safely:
String[] words = str.trim().split("\\s+");
```

- High-Performance Random Number Generation:

```
// Avoid creating new Random() instances; use ThreadLocalRandom:
int randInt = ThreadLocalRandom.current().nextInt(min, max + 1); // [min, max] inclusive
```

14 The PriorityQueue In-Place Mutation Trap (Dijkstra & Lazy Deletion)

A lethal trap in graph algorithms (e.g., Dijkstra's or Prim's algorithms) is attempting to mutate an object that already resides inside a PriorityQueue:

```
// FATAL BUG: Mutating an element inside a heap DOES NOT trigger re-sifting!
Node u = map.get(nodeId);
u.distance = 5; // Heap array is now CORRUPTED! The root does NOT rebalance!
```

Java's PriorityQueue does not track element state changes. Calling pq.remove(u) takes $O(N)$ linear scan time, destroying Dijkstra's $O((V + E) \log V)$ time complexity!

14.1 The Production Solution: The Lazy Deletion Pattern

Instead of updating existing objects, simply insert a **new entry** into the heap. When polling from the heap, discard stale outdated entries in $O(1)$:

```
// Dijkstra's Lazy Deletion Template in Java
PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));
int[] dist = new int[V];
Arrays.fill(dist, Integer.MAX_VALUE);
dist[src] = 0;
pq.offer(new int[]{src, 0});

while (!pq.isEmpty()) {
    int[] curr = pq.poll();
    int u = curr[0];
    int d = curr[1];

    // LAZY DELETION: If we already found a shorter path to u, skip this stale entry!
    if (d > dist[u]) continue;

    for (int[] edge : adj.get(u)) {
        int v = edge[0], weight = edge[1];
        if (dist[u] + weight < dist[v]) {
            dist[v] = dist[u] + weight;
            pq.offer(new int[]{v, dist[v]}); // Push fresh entry; old entry lazily skipped later
        }
    }
}
```

15 The Index-Array Sorting Pattern (Zero-Allocation Parallel Sort)

Suppose you are given two separate arrays: int[] startTimes and int[] endTimes. You need to sort the intervals by startTimes, but creating a dedicated class Interval introduces memory allocation and garbage collection overhead.

The Competitive Programming Secret: Sort the Indices!

```
int n = startTimes.length;
Integer[] idx = new Integer[n];
for (int i = 0; i < n; i++) idx[i] = i;

// Sort indices based on startTimes; tie-break on endTimes:
Arrays.sort(idx, (i, j) -> {
    if (startTimes[i] != startTimes[j]) {
        return Integer.compare(startTimes[i], startTimes[j]);
    }
    return Integer.compare(endTimes[i], endTimes[j]);
});

// Now iterate in sorted order without allocating a single object:
for (int i : idx) {
    int s = startTimes[i];
    int e = endTimes[i];
    // Process interval [s, e]
}
```

16 The Three Classic Numeric & Overflow Traps in Java

1) Binary Search Midpoint Overflow: Writing mid = (low + high) / 2 causes integer overflow if low + high $> 2^{31} - 1$, becoming a negative number!

```
// SAFE ALTERNATIVE 1:
int mid = low + (high - low) / 2;
// SAFE ALTERNATIVE 2 (Logical unsigned shift):
int mid = (low + high) >>> 1;
```

2) The Math.abs(Integer.MIN_VALUE) Trap: Because 32-bit signed integers span $[-2^{31}, 2^{31}-1]$, the positive equivalent of $-2{,}147{,}483{,}648$ does not fit in an int! Math.abs(Integer.MIN_VALUE) **returns negative $-2{,}147{,}483{,}648$!**

```
// FIX: Cast to long BEFORE taking absolute value:
long safeAbs = Math.abs((long) val);
```

3) Large Modulo Multiplication Overflow: Writing (a * b) % MOD when $a, b \approx 10^9$ causes $a \times b \approx 10^{18}$, which overflows 32-bit integer limits before the modulo is applied:

```
// FIX: Cast the first operand to long:
int ans = (int) (((long) a * b) % MOD);
```

17 Iconic String Comparator Tricks: LeetCode 179 "Largest Number"

When asked to assemble an array of non-negative integers into the largest possible number (e.g., [3, 30, 34, 5, 9] $\implies$ "9534330"):

- Standard numeric comparison fails because 3 should appear before 30 ($330 > 303$).
- The Concatenation Comparator: Compare $(b + a)$ against $(a + b)$:

```
public String largestNumber(int[] nums) {
    String[] strs = new String[nums.length];
    for (int i = 0; i < nums.length; i++) strs[i] = String.valueOf(nums[i]);

    // Transitive Concatenation Comparator:
    Arrays.sort(strs, (a, b) -> (b + a).compareTo(a + b));

    // Handle corner case where array contains only zeros: [0, 0] -> "0"
```

```
    if (strs[0].equals("0")) return "0";

    return String.join("", strs);
}
```

18 The PriorityQueue Iteration Trap (Why for-each is NOT Sorted)

A widespread misconception among intermediate Java developers is assuming that iterating over a PriorityQueue visits elements in sorted order:

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.addAll(Arrays.asList(5, 1, 9, 3));

// CATASTROPHIC BUG: Iteration DOES NOT print 1, 3, 5, 9!
for (int num : pq) {
    System.out.print(num + " "); // Prints internal array layout: e.g., 1 3 9 5 !
}
```

Why Does This Happen? A PriorityQueue is backed by a binary heap array satisfying only the Heap-Invariant (parent $\leq$ children), not the complete binary search tree invariant. The iterator() simply traverses the underlying raw array indices $0, 1, \ldots, N-1$.

The Iron Rule: To extract elements in strict sorted order, you must drain the heap via poll():

```
while (!pq.isEmpty()) {
    System.out.print(pq.poll() + " "); // Guaranteed strictly sorted: 1 3 5 9
}
```

19 The 3-Line Built-In LRU Cache (LinkedHashMap)

While interviewers sometimes ask you to code an LRU Cache from scratch using a Doubly Linked List and HashMap (covered in Volume 2), in algorithmic problems (or when allowed), Java has an LRU Cache **natively built into the standard library**:

```
import java.util.*;

public class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;

    // true sets 'access-order' (LRU) instead of default 'insertion-order'
    public LRUCache(int capacity) {
        super(capacity, 0.75f, true);
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity; // Automatically evicts least-recently used entry!
    }
}
```

20 The Immutability Trap: List.of() vs. Arrays.asList() vs. new ArrayList<>()

Passing lists to graph and backtracking methods requires knowing their mutation capabilities:

TABLE 2
Java List Instantiation Mutability Comparison

Factory Method	Allow add() / remove()?	Allow set(i, val)?	Null Elements?
List.of(1, 2)	No (UnsupportedOperation)	No (UnsupportedOperation)	Throws NPE
Arrays.asList(arr)	No (Fixed-size array wrapper)	Yes (Mutates underlying array)	Permitted
new ArrayList<>(...)	Yes (Fully mutable)	Yes (Fully mutable)	Permitted

Interview Rule: If you intend to add or remove elements later, always wrap with new ArrayList<>():

```
List<Integer> mutable = new ArrayList<>(Arrays.asList(1, 2, 3));
```

21 Greatest Common Divisor (GCD), LCM, and Fast Modular Power

21.1 1. Euclidean GCD and LCM in 3 Lines

```
// Greatest Common Divisor (Euclidean Algorithm): O(log(min(a, b)))
public static long gcd(long a, long b) {
    return b == 0 ? a : gcd(b, a % b);
}

// Least Common Multiple
public static long lcm(long a, long b) {
    return (a / gcd(a, b)) * b; // Divide first to avoid arithmetic overflow!
}

// Or via built-in BigInteger:
long builtInGcd = BigInteger.valueOf(a).gcd(BigInteger.valueOf(b)).longValue();
```

21.2 2. Fast Modular Exponentiation (Binary Power: $O(\log N)$)

Calculating $(x^n) \pmod{\text{MOD}}$ in $O(\log n)$ instead of $O(n)$:

```
public static long modPow(long base, long exp, long mod) {
    long res = 1;
    base %= mod;
    while (exp > 0) {
        if ((exp & 1) == 1) res = (res * base) % mod;
        base = (base * base) % mod;
        exp >>= 1;
    }
    return res;
}
```

22 The 8-Line Fenwick Tree (Binary Indexed Tree / BIT)

For dynamic range sum queries with $O(\log N)$ point updates (LeetCode 307), a full Segment Tree is 50 lines of code. A **Fenwick Tree** achieves identical performance in 8 lines using the two's-complement low-bit trick (i & -i):

```
public class FenwickTree {
    private final int[] tree;
    public FenwickTree(int n) { tree = new int[n + 1]; } // 1-indexed

    public void update(int i, int delta) {
        for (; i < tree.length; i += i & -i) tree[i] += delta;
    }

    public int query(int i) { // Prefix sum from 1 to i
        int sum = 0;
        for (; i > 0; i -= i & -i) sum += tree[i];
        return sum;
    }
}
```

23 The Array-Based Trie (Prefix Tree) Archetype

In dictionary search, autocomplete, and Boggle/Word Search II (LeetCode 212), avoid heavy HashMap child nodes. Use an array of 26 pointers:

```
public class TrieNode {
    public TrieNode[] children = new TrieNode[26];
    public boolean isWord = false;
    public String word = null; // Storing complete word at leaf eliminates backtrack string rebuilding!
}

public void insert(TrieNode root, String word) {
    TrieNode curr = root;
```

```
    for (char c : word.toCharArray()) {
        int idx = c - 'a';
        if (curr.children[idx] == null) curr.children[idx] = new TrieNode();
        curr = curr.children[idx];
    }
    curr.isWord = true;
    curr.word = word;
}
```

24 In-Place Matrix Transformations (Rotate 90 Degrees Clockwise)

To rotate an $N \times N$ matrix by 90 degrees clockwise in-place with $O(1)$ extra memory (LeetCode 48):

1) Transpose: Swap matrix[i][j] with matrix[j][i] across the main diagonal.
2) Reverse Rows: Invert each row horizontally.

```
public void rotate(int[][] matrix) {
    int n = matrix.length;
    // 1. Transpose
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            int temp = matrix[i][j]; matrix[i][j] = matrix[j][i]; matrix[j][i] = temp;
        }
    }
    // 2. Reverse each row
    for (int i = 0; i < n; i++) {
        for (int j = 0, k = n - 1; j < k; j++, k--) {
            int temp = matrix[i][j]; matrix[i][j] = matrix[i][k]; matrix[i][k] = temp;
        }
    }
}
```

25 The Zero-Off-By-One Predicate Binary Search Archetype

Candidates frequently introduce infinite loops with while (low <= high) by mismanaging ±1. The **Predicate Binary Search template** solves all "Binary Search on Answer" problems (e.g., Koko Eating Bananas, Capacity To Ship Packages) with zero edge-case bugs:

```
// Find the MINIMUM value that satisfies condition(mid) == true
int low = minPossible, high = maxPossible;

while (low < high) {
    int mid = low + (high - low) / 2;
    if (isValid(mid)) {
        high = mid;     // Target is at mid or to the left (don't discard mid!)
    } else {
        low = mid + 1;  // mid is definitely invalid; look right
    }
}
return low; // low == high is guaranteed to be the exact minimal valid answer!
```

26 Fast I/O in Java: Eliminating Scanner Time Limit Exceeded (TLE)

In technical screens or competitive platforms reading 10^5+ inputs, Scanner uses slow regex tokenization that causes TLE. Use BufferedReader and StringTokenizer:

```
import java.io.*;
import java.util.*;

public class FastIO {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out = new PrintWriter(new BufferedOutputStream(System.out));
        StringTokenizer st = null;
```

```
        String line = br.readLine();
        if (line != null) st = new StringTokenizer(line);

        int val = Integer.parseInt(st.nextToken());
        out.println(val);
        out.flush(); // Crucial: Flush buffer before exit!
    }
}
```

27 Summary: The 5-Second Decision Checklist

Before writing any comparator in an interview or project, follow this 3-step mental rubric:

1) Are you sorting primitives or objects?
   - If 2D primitive array (int[][]): Use Comparator.comparingInt(a -> a[idx]).
   - If custom object: Use method reference Comparator.comparingInt(Class::getVal).
2) Do you need a Min-Heap or a Max-Heap?
   - Min-Heap: Compare A to B (a -> a[0]).
   - Max-Heap: Reverse it ((a, b) -> Integer.compare(b[0], a[0]) or append .reversed()).
3) Are there tie-breakers?
   - Write the explicit ternary lambda: (a, b) -> a.val != b.val ? compareVal : compareTieBreaker.